found and deleted in this branch, and that `childNode` currently violates Ss-tree's invariants, so its parent needs to do some fixing.

#11 To that extent, save current child in the temp variable we had previously initialized…

#12 If a point has been deleted in current node's subtree, then exit the `for` loop (we assume there are no duplicates in the tree, so a point can be in one and one branch only).

#13 Check if none of `node`'s children violates Ss-tree's invariants: in that case we won't need to do any fix for current node.

#14 However, if the point was deleted in this subtree, we still need to re-compute the bounding envelope.

#15 Then we can return, letting the caller know if the point was deleted as part of this call, but that this node doesn't violate any invariant.

#16 If, instead, one of current node's children does violate an invariant, as the result of calling `delete` on it, the first thing we need to do is retrieving a list of the sibling of that children (stored in `nodeToFix`), but filtering in only those that in turn have more than `m` children/points: we will try to move one of those entries (either children or points, for internal nodes and leaves respectively) from one of the siblings to the one child from which we deleted `target`, and that now has too few children.

#17 Check if there is any sibling of `nodeToFix` that meets the criteria.

#18 If `nodeToFix` has at least one sibling with more than `m` entries, move one entry from one of the siblings to `nodeTofix` (which will now be fixed, because it will have exactly `m` points/children).